From Vector Search to Hybrid Retrieval

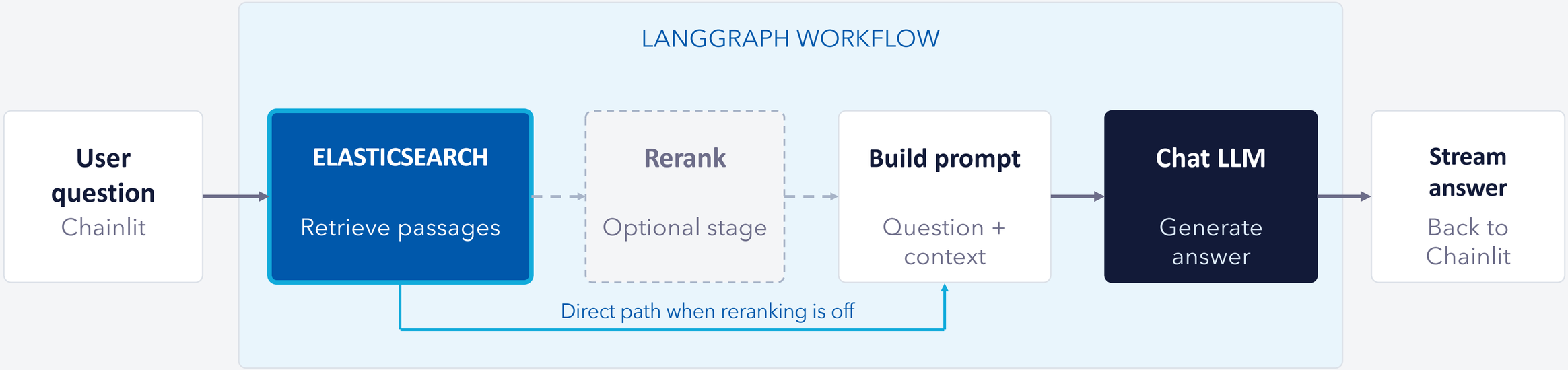
Enhancing RAG with Elasticsearch



Emir Can
Senior AI Engineer
sogeti
Part of Capgemini

Generic RAG: where Elasticsearch fits

Chainlit handles the conversation. LangGraph coordinates retrieval and generation.



Measured in this talk

BM25, vector and hybrid / RRF

Downstream

Generated answers are not scored.

Elasticsearch runs in the standalone retrieval comparison. Chainlit integration is conceptual; reranking is currently disabled.

Three ways to ask for the same knowledge

01 **Exact**

index.lifecycle.rollover_alias

A precise setting name

02 **Symptom**

“Every string is indexed in two forms.”

Describe behavior without naming the mechanism

03 **Mixed**

ILM ERROR step + inspect and retry

A technical clue combined with a task

18 evaluation questions: 6 exact, 6 symptom/semantic and 6 mixed.

Three retrieval modes

01

Lexical

BM25

Search analyzed terms in passage text.

Which passages match these words?

02

Vector

kNN

Embed the question. Search nearby passage vectors.

Which passages appear semantically similar?

03

Hybrid

BM25 + kNN with RRF

Run both retrievers. Combine their ranked lists.

How should we combine these retrieval signals?

RRF is the fusion method inside hybrid. The embedding model represents text; it does not grade relevance.

Why Elasticsearch: text and vectors together

Docling

Extract document content

Split passages

1,000-character target
200-character overlap

Embedding model

Application embeds passages

Elasticsearch

Index text, vectors + metadata

One stored passage

```
content: "When disk usage exceeds the flood-stage watermark..."
content_vector: [0.013, -0.027, ...]
metadata:
   filename: fix-watermark-errors.html
   source_url: ...
_id: stable passage ID
```

One index, three paths

1. BM25
2. Vector / kNN
3. Hybrid / native RRF

The expanded experiment

24

official articles

303

indexed passages

18

evaluation questions

18 questions × 3 modes × 10 passages = 540 returned slots

Fixed corpus, question set, embeddings, relevance rubric and final cutoff.

36-question inventory: 18 evaluation questions graded, 18 development questions remain unreviewed.

Where relevance judgments come from

Question + passage → Assign a relevance grade → Fixed reference labels

EXPERT REVIEW

Human gold standard

Experts apply an agreed rubric to question–passage pairs.

Resolve disagreements.
Audit evidence coverage.

AI-ASSIGNED LABELS HERE

LLM-as-judge

A model grades passages using a question and rubric.

Check agreement with humans.
Our labels are saved beforehand.

BEHAVIOR IN USE

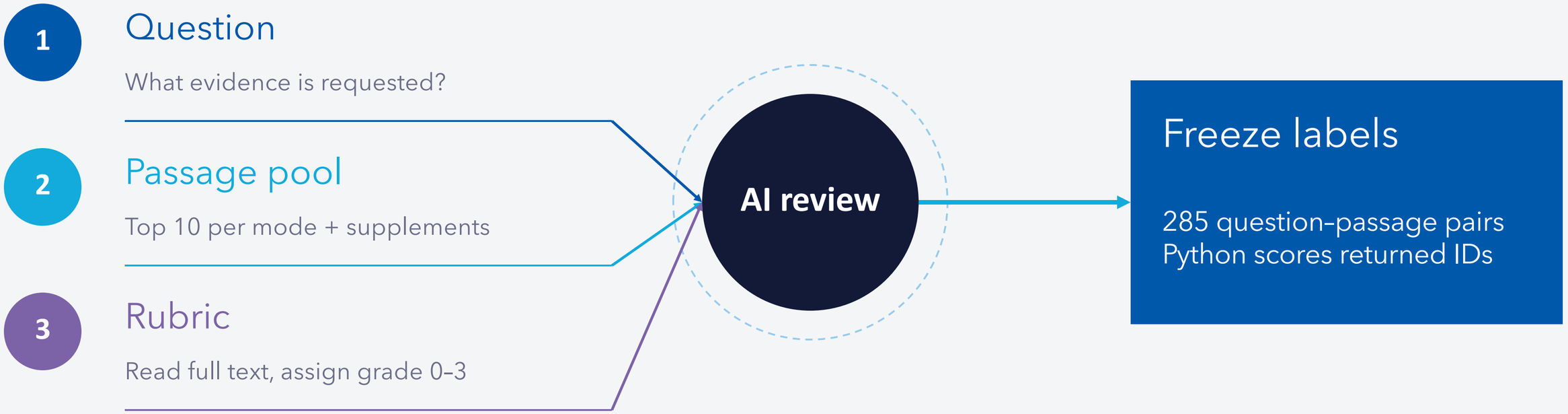
User proxies

Clicks, feedback, repeat queries and task completion.

Account for position bias.
Clicks do not label unseen evidence.

The metric formulas stay the same. What changes is how the reference judgments are obtained.

Our setup: AI review, then fixed-label scoring



3	Requested fact or step
2	Useful support
1	Marginal overlap
0	No requested evidence

Useful = grade 2 or 3. No runtime judge call. Human audit pending. Recall and ideal nDCG use the graded pool.

Metrics

01

Precision@10

useful in top 10 / 10

How much returned context is useful?

02

Recall@10

useful in top 10 / useful in pool

How much known useful evidence is found?

03

RR@10

1 / first useful rank

How early does useful evidence appear?

04

MRR@10

(RR_1 + ... + RR_18) / 18

First-evidence ranking across questions

nDCG rewards useful evidence earlier

Useful = grade ≥2. RR is 0 when no useful passage appears in the top 10. Aggregate scores average questions

.

Recorded run: one question through three modes

Question → 3 retrieval modes → Passage IDs → Fixed grades → Metrics

```
for mode in ("bm25", "vector", "hybrid"):
    hits = store.search_with_scores(
        case["query"], k=10, mode=mode)

ids = [doc.id for doc, _ in hits]

metrics = ranking_metrics(
        ids, case["relevance"],
        k=10, relevance_threshold=2)
```

54

question–mode runs

540

returned slots

Repeat for 18 questions and average each metric. Excerpt adapted from the implementation; recorded results follow.

Demo

Same coverage, different ordering

ILM: rollover_alias does not point to the index. Which alias and bootstrap settings should I inspect?



Circles show relevance grades at ranks 1–10. Grades 2/3 are useful. RR@10 = 1.000 for all three modes.

Vector leads on context quality at top 10

BM25

0.714

nDCG@10

Precision@10	30.0%
Pool recall@10	68.8%
MRR@10	0.889

Vector / kNN

0.850

nDCG@10

Precision@10	39.4%
Pool recall@10	89.1%
MRR@10	0.926

Hybrid / RRF

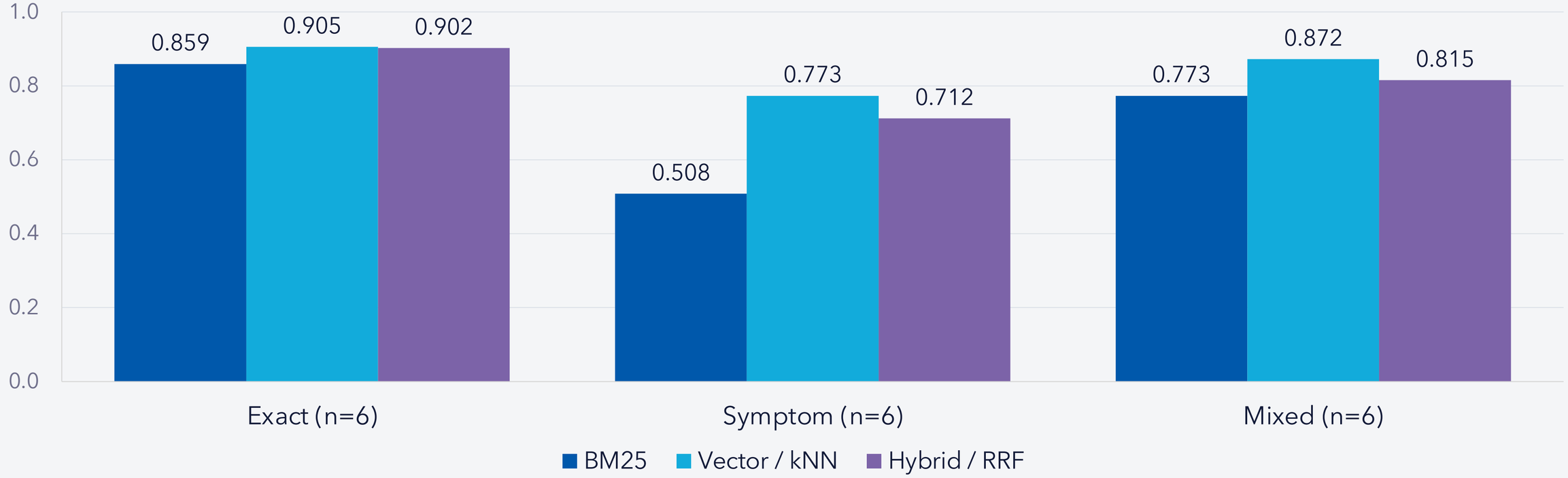
0.810

nDCG@10

Precision@10	37.8%
Pool recall@10	86.9%
MRR@10	0.944

Means over 18 questions. Vector leads nDCG, precision and pool recall; hybrid leads MRR. AI labels await human audit.

Query categories explain where results differ

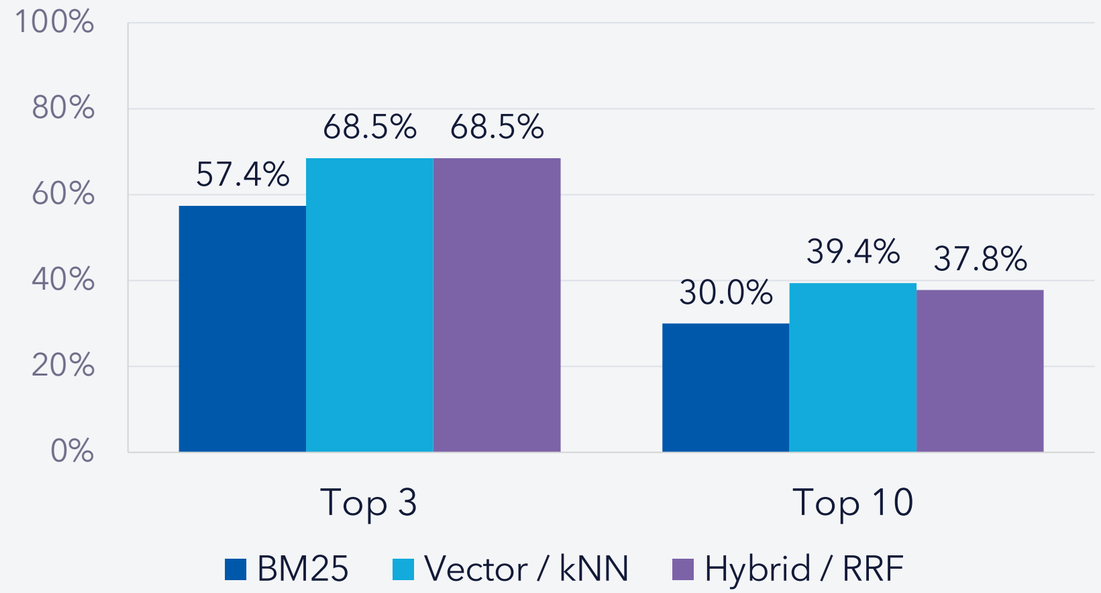


Vector leads on nDCG in each category in this run.

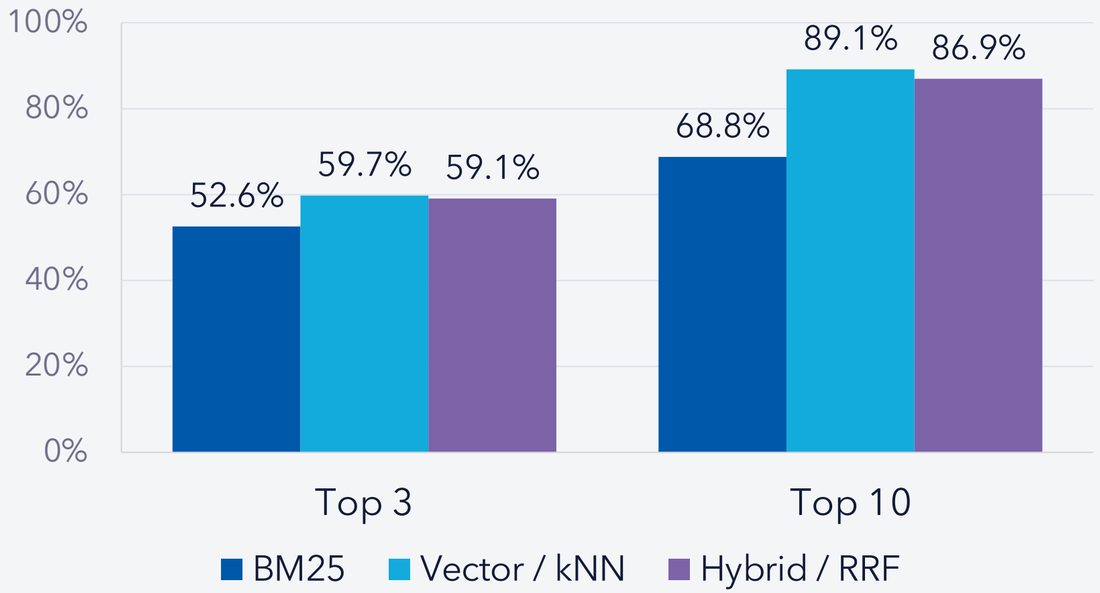
Exact: vector 0.905 and hybrid 0.902 are very close. Small category samples remain descriptive.

More evidence, more noise in the returned context

Precision



Recall



Precision falls as more passages are returned. Coverage of known useful evidence rises.

Same expanded labels. nDCG @3 / @10: BM25 0.722 / 0.714, vector 0.792 / 0.850, hybrid 0.793 / 0.810.
Changing k also changes vector k and the hybrid candidate window.

THE TAKEAWAY

Choose the objective before the winner.

Context quality here: vector
First useful evidence: hybrid

Next: human audit, fresh questions and generated-answer evaluation.